

VECTORIZATION

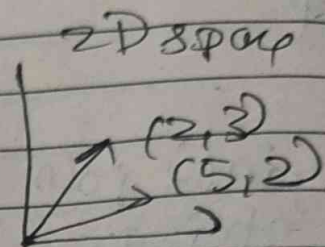
Date: / / Page no:

Similarity b/w tags.

new-df [tags] [0]
[1]

Similarity score b/w text

Tags $\rightarrow$ Vectors.
(Text) $\rightarrow$ Vectors



* Bag of words technique use.
 $\rightarrow$ word2vec

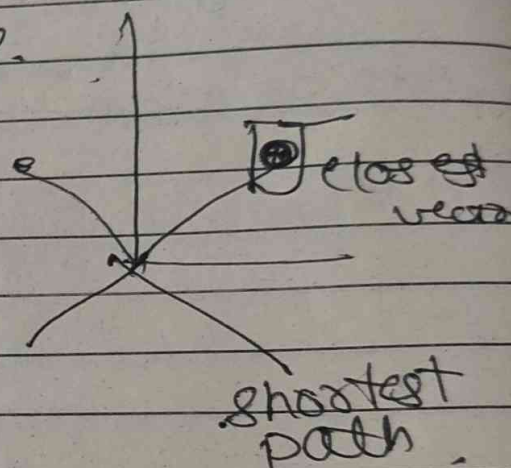
All tags combines (concatenate)

+
tags 1 + tags 2 + ...
(large text)

$\rightarrow$ 5000 most common words

+
5000 most common words

	w_1	w_2	...	(w_{5000})	
m_1	5	3		1	} vectors 5000, 800
m_1	Action	feature			
m_2					
m_3					



In Vectorization do not use stop words
as they are common

→ don't have meaning

Using → Scikit learn
library → count vectorizer.
class

from sklearn.feature_extraction.text
import CountVectorizer

cv = CountVectorizer(max_features=5000,
stop_words='english')

* Scikit-learn (sklearn) → ML libraries.
It provides ready made also like.

- Classification
- Regression
- Clustering
- Recommendation
- Feature Engin...

- It is a toolbox
- KNN
- Linear regression
- Decision...

* Feature extraction. → converting text
into numerical feature

* Count Vectorizer - convert text into a
matrix of word count

max-features = 5000
If dataset has 1,20,000 words then
only 5000 most frequent words

→ cv.fit-transform(news_df['tags'])

→ fit means learn from data

→ transform → converted into vector
Ex - action hero figure
(1, 1, 10, 0)

cv.get_feature_names_out()

Issue → 2 similar words
activity and digits
activity and digits
only one

Apply stemming method.

['loved', 'loving', 'love']

['love', 'love', 'love']

we use nlp library nltk.

ps. stem ('loving')
('loved')

→ Out → love.

Stemming in NLP

import nltk

from nltk.stem.porter import PorterStemmer

ps = PorterStemmer()

def stem(text):

y = []

// create empty list

for i in text.split():

y.append(ps.stem(i))

return " ".join(y)

NLP - field of AI that enables computer to understand, process, analyse and generate human language.

Stemming:- Stemming is the process of reducing a word to its root (stem) form by removing prefix or suffixes.

Ex $\Rightarrow$ loving $\rightarrow$ love

Lemmatization uses grammar and dictionary

NLTK $\rightarrow$ Natural Language Toolkit

provides tools for tokenization, stemming, lemmatization, stop words

* Porter Stemmer

→ oldest and famous (widely used)

stemming algo

* `ps = PorterStemmer()` create object

* what `split()` do.

Ex `I love watching Avatars` = test
`text.split()`

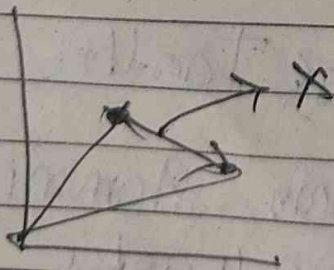
`['I', 'love', 'watching', ...]`

Next step.

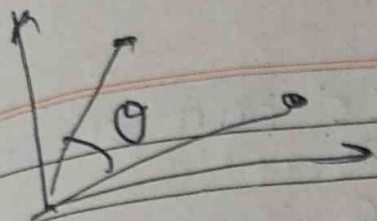
We have to calculate distance of a movie from another movie so that

$$\text{distance} \propto \frac{1}{\text{similarity}}$$

We are not calculating Euclid distance



We are calculating cosine distance



Angle α similarity

Angle less similarity more

* from sklearn.metrics.pairwise
import cosine_similarity

cosine_similarity(vectors)

1 movie not 4806
2 movie or

* sklearn.metrics has metrics module
inside sklearn that provide f^n to measure
or evaluate relationship b/w data.

* It contains many metrics like:

→ Accuracy score
→ Precision
→ Recall
→ F1 score

→ Mean square error
→ Cosine similarity
→ ...

* Pairwise means:- compare every pair
of objects.

Compare every pair of objects.
Suppose we have

Movie A

Movie B

Movie C

Pairwise comparisons

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow C$

if there are n movies.

then pairwise comparison compares every
movie with every other movie.

Mathematical Formula

Cosine Similarity

$\frac{A \cdot B}{\|A\| \|B\|}$

$\frac{A \cdot B}{\|A\| \|B\|}$

pseudo →
recommended

movie dego → 5 similar movie
movie input

↓
movie have [index]

↓
similarity matrix of that movie

↓
avatar[0]

↓
distance of avatar will all movies

↓
sort

→ highest to lower
similar

def recommended(movie):
return

How to do masking

any new_df[new_df['title'] == 'Avatar']

Output

	movie	title
0	19995	Avatar

key


```
def recommend(movie):
```

```
    movie_index = new_df[new_df['title'] == movie].index[0]
```

```
    distances = similarity(movie_index)
```

```
    movies_list = sorted(list(enumerate(movies)))
```

```
    for i in movies_list:
```

```
        print(new_df.iloc[i[0]]['title'])
```

def → define a funⁿ.

return title

index ko use krni because if movie index n
 baaki movie ki cosine similarity distance
 find krne ke liye

Python generally sort in asc order
 but to get high similarity

(distances)), reverse=True, key=lambda x: x[1][1])

enumerate is used so that when
 sorting it got index out of

and if don't use it use get

Output

Indices lost

0.81 sort

0.81

0.25

0.73

0.73

0.25

Enumerate

(0, 0.81)
 (2, 0.73)
 (1, 0.25)

Key = lambda x: x[1]

Ex -> student = [
 ('Anadi', 90)
 ('Rajat', 98)
 ('M21', 95)]

By name

Anadi

M21

Rajat

Now python ignore
 first elements and
 start using the
 second element

sorted (students)